MultiGPUs y modelos híbridos

Universidad Nacional de La Plata



Facultad de Informática

Prof. Dr. Adrian Pousa



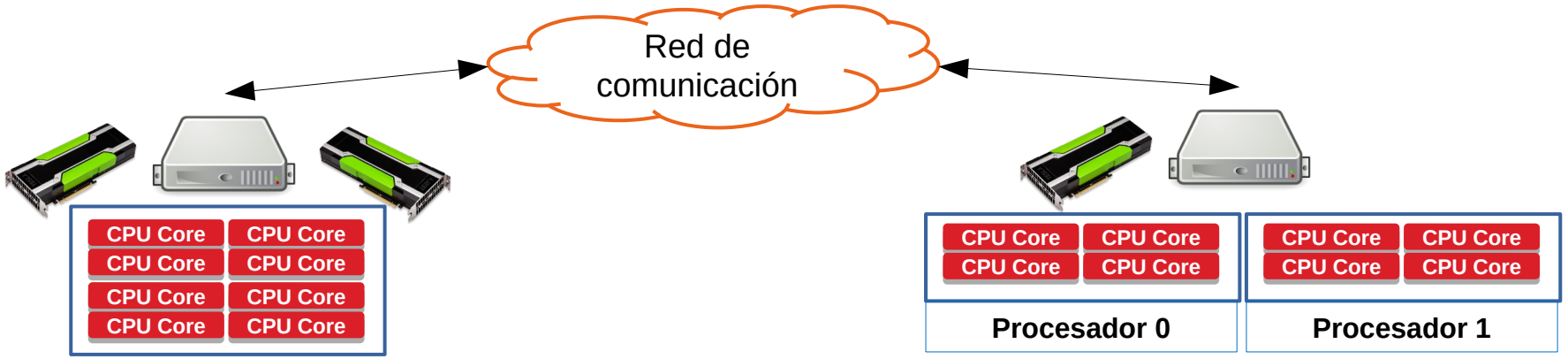
1. **Introducción a los modelos híbridos**
2. **Arquitecturas HPC y software de base**
   Manycores y Múltiples GPUs. Multicores: OpenMP y Pthreads. Cluster: MPI.
3. **Modelos híbridos**
   CUDA + OpenMP. CUDA + Pthreads. CUDA + MPI. CUDA + OpenMP + MPI / CUDA + Pthreads + MPI.
4. **Modularidad**
5. **Distribución de carga**



1 Introducción a los modelos híbridos

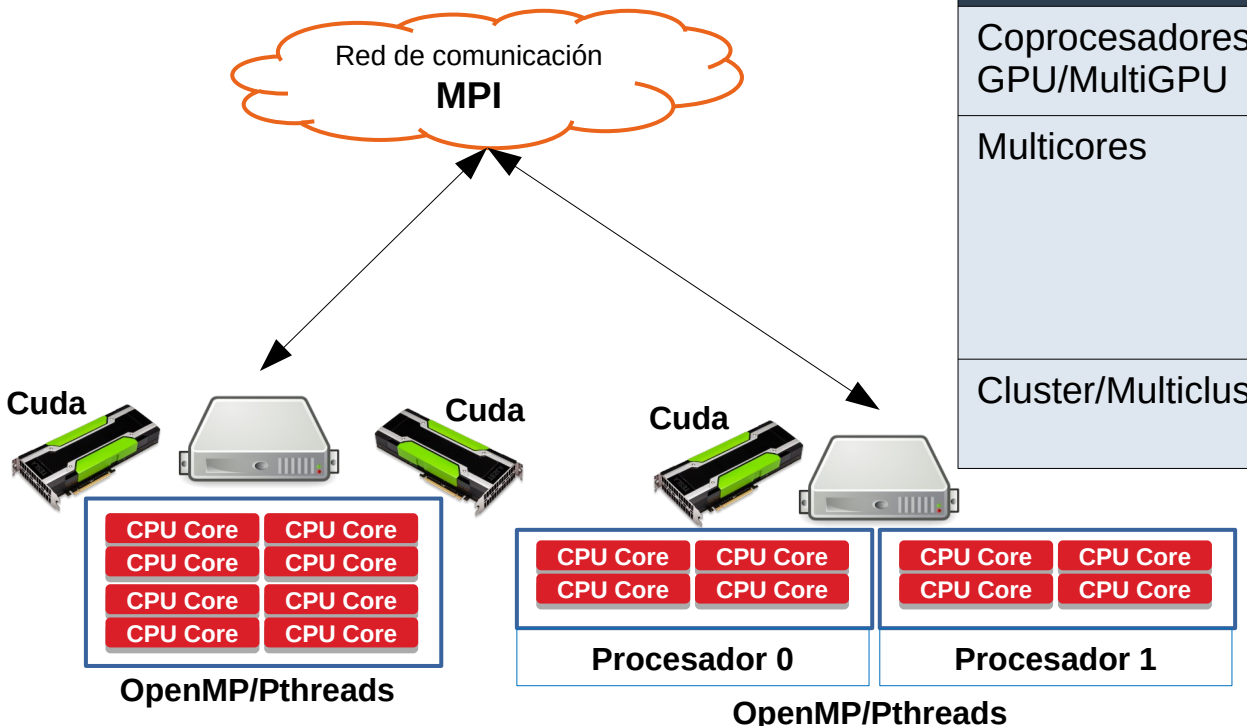
Introducción a los modelos híbridos

- En la actualidad existe una diversa cantidad de hardware muy potente:
  - Procesadores tradicionales (Intel, AMD, ARM etc) con múltiples cores
  - Manycores/GPUs
    - Servidores de alto rendimiento pueden incluir hasta 4 GPUs
- Las redes de comunicación permiten unir toda esta potencia de cómputo en Cluster/Multicluster formando un modelo híbrido.



Introducción a los modelos híbridos

- Podemos considerar 3 niveles de hardware. Cada nivel requiere una herramienta específica para programar la arquitectura subyacente.



Hardware	Herramienta
Coprocesadores/Manycores GPU/MultiGPU	Cuda, OpenCL, Sycl etc.
Multicores	Modelo de memoria compartida: • OpenMP • Pthreads Modelo de memoria distribuida: • MPI
Cluster/Multicluster	Modelo de memoria distribuida: • MPI

Introducción a los modelos híbridos

- Programar sobre un modelo híbrido conlleva complejidad que se presenta al integrar el hardware y el software.
- Existen algunas herramientas que nos dan mayor nivel de abstracción y facilitan la programación:
  - OpenACC
  - Intel OpenAPI
- Pero si queremos tener control absoluto debemos programar sin abstraernos.
- Para un correcto desarrollo se deben tener en cuenta distintos aspectos:
  - Compilación sobre arquitecturas diferentes
  - Compilación en módulos separados para cada arquitectura
  - Enlazar los módulos de cada arquitectura
  - Integrar diversas opciones de compilación



2

Arquitecturas HPC y software de base

- *Manycores y Múltiples GPUs*
- *Multicores*
  - *OpenMP*
  - *Pthreads*
- *Cluster: MPI*



2

Arquitecturas HPC y software de base

Manycores y Múltiples GPUs

Multicores

OpenMP

Pthreads

Cluster: MPI

Manycores - Múltiples GPUs

- En una máquina con más de una GPU debemos especificar sobre que GPU se están realizando las acciones. Para esto, se utiliza la sentencia:

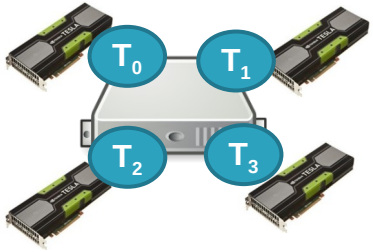
```
cudaSetDevice(int id_ device);
```

  - `id_device`: número entero entre 0 y el número de GPUs - 1 instaladas.

```
...
cudaSetDevice(0); //Todo lo que se haga de acá en más: sobre GPU0
cudaMalloc(…);
cudaMemcpy(…);
Kernel_0<<<..,..,..>>>();
...
cudaSetDevice(1); //Todo lo que se haga de acá en más: sobre GPU1
cudaMalloc(…);
cudaMemcpy(…);
Kernel_1<<<..,..,..>>>();
...
```

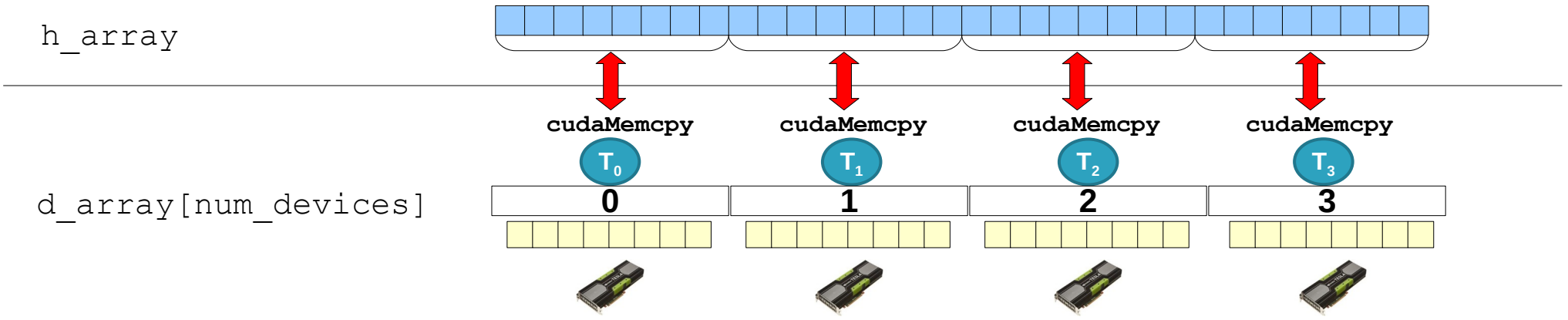
Manycores - Múltiples GPUs

- Para realizar el cómputo correcto de un conjunto de datos utilizando varias GPUs, se suele crear un hilo por cada GPU disponible.
- El identificador de cada hilo debería coincidir con el `id_device`.
- Cada hilo se encarga de gestionar todo lo relacionado con su propia GPU.



Manycores - Múltiples GPUs

- Ejemplo: Se quiere realizar cómputo sobre N elementos almacenados en un array.
  - El cómputo de una porción del array es independiente del resto.
  - Suponemos `num_devices` GPUs iguales, los datos se distribuyen proporcionalmente entre ellas (porción = N/`num_devices`)
  - Las porciones del array en GPU conviene gestionarlas mediante un array indexado por `id_device` (o identificador del hilo).
  - Cada hilo debe calcular la posición del array a copiar en su porción en GPU.





2

Arquitecturas HPC y software de base

- *Manycores y Múltiples GPUs*
- *Multicores*
  - *OpenMP*
  - *Pthreads*
- *Cluster: MPI*

Multicores

- Habitualmente, el modelo de programación en memoria compartida utiliza dos herramientas:
  - Posix threads (Pthread):
    - Adecuado para paralelismo funcional
    - Mayor control sobre los hilos
  - OpenMP:
    - Modelo *Fork-Join*
    - Adecuado para paralelismo de datos
    - Gestión de hilos implícita



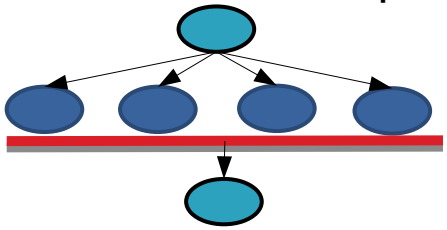
2

Arquitecturas HPC y software de base

- *Manycores y Múltiples GPUs*
- *Multicores*
  - *OpenMP*
  - *Pthreads*
- *Cluster: MPI*

Multicores - OpenMP

- **OpenMP** es una API que extiende los lenguajes Fortran, C y C++ con directivas para paralelizar código en arquitecturas de memoria compartida.
- Utiliza el modelo fork-join.



- Cada directiva tiene la forma:

```
pragma omp <directiva> [cláusula [ , ...] ...]
```

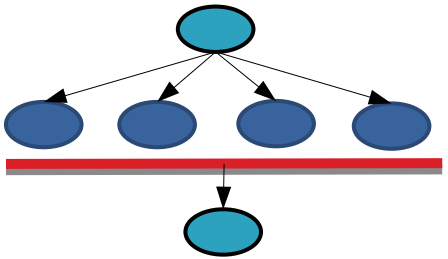
- Incluida en el compilador GCC:
  - Cabecera de programa C: `#include<omp.h>`
  - Compilación: `gcc -fopenmp archivoFuente.c`

Multicores - OpenMP

- Ejemplo de paralelización de un for:

```
int main(int argc, char** argv){
...
  #pragma omp parallel for
      for (i=1; i<n; i++) // i privada para cada hilo por defecto
           b[i] = (a[i] + a[i-1]) / 2.0;
...
}
```

  - La directiva crea tantos hilos como unidades de procesamiento (*fork*).
  - Las iteraciones del `for` se dividen entre los hilos.
  - Al final del `for` los hilos hacen una barrera implícita (*join*).





2

Arquitecturas HPC y software de base

- *Manycores y Múltiples GPUs*
- *Multicores*
  - *OpenMP*
  - *Pthreads*
- *Cluster: MPI*

Multicores - Pthreads

- Pthread (Posix threads - estándar Posix 1.0c 1995) API para programación multihilo sobre memoria compartida multiplataforma
- Posee un conjunto de funciones que permiten:
  - Gestionar de hilos (threads)
  - Sincronizar de los hilos: Mutexes, Variables condición, Barreras
- Incluida en el compilador GCC:
  - Cabecera de programa C: `#include<pthread.h>`
  - Compilación: `gcc -pthread fuente.c -o ejecutable`

Multicores - Pthreads

- Los hilos pueden definirse:
  - Individualmente: `pthread_t miHilo;`
  - Cómo arreglos de hilos: `pthread_t misHilos[T];`
- Luego deben crearse explícitamente mediante la función:

```
int pthread_create(pthread_t *thread, const pthread_attr_t *attr, void *(*start_routine)(void*), void *arg);
```

Parámetros:

- **thread:** hilo definido con pthread_t.
- **attr:** atributos de configuración del hilo a crear.
- **start_routine:** puntero a la función que implementa el comportamiento del hilo.
- **arg:** argumento pasado como parámetro a la función anterior.

Valor de retorno:

- **pthread_create** retorna **cero** si tiene **éxito**, sino retorna un código de error.

Multicores - Pthreads

- Pthreads provee sentencias para:
  - Sincronización por Exclusión mutua:
    - `pthread_mutex_lock()`
    - `pthread_mutex_unlock()`
  - Barreras
    - `pthread_barrier_wait()`
  - Sincronización por condición:
    - `pthread_cond_wait()`
    - `pthread_cond_signal()`



2

Arquitecturas HPC y software de base

- *Manycores y Múltiples GPUs*
- *Multicores*
  - *OpenMP*
  - *Pthreads*
- *Cluster: MPI*

Cluster - MPI

- **MPI** es una especificación para implementar el modelo de programación sobre memoria distribuida (pasaje de mensajes).
- Puede ser usada tanto en arquitecturas de memoria distribuida (más habitual) como de memoria compartida.
- Existen varias distribuciones implementadas en Fortran, C y C++:
  - OpenMPI
  - MPICH
  - Etc.
- No viene como parte de GCC (debe instalarse)
- Cabecera de programa C: `#include <mpi.h>`
- Compilación básica (openMPI): `mpicc archivoFuente.c`

Cluster - MPI

- La ejecución de un programa MPI se suele hacer con comandos como `mpirun` o `mpiexec` según la distribución.
- Ejemplo de ejecución:

```
mpirun -np 3 -host proc1,proc1,proc2 programa
```

  - La línea anterior ejecuta el programa con 3 procesos (NO Hilos):
    - 2 procesos al procesador proc1
    - 1 proceso al procesador proc2

Cluster - MPI

- MPI permite el envío y recepción de mensajes punto a punto, sincrónico o asincrónico.
- También provee operaciones colectivas que permiten distribuir múltiples datos entre varios procesadores con una única sentencia.

Operaciones punto a punto	Operaciones colectivas
MPI_Send MPI_Isend MPI_Recv MPI_Irecv	MPI_Bcast MPI_Gather MPI_Scatter MPI_Gatherv MPI_Scatterv



3

Modelos híbridos

- *CUDA + OpenMP*
- *CUDA + Pthreads*
- *CUDA + MPI*
- *CUDA + OpenMP + MPI / CUDA + Pthreads + MPI*

Modelos híbridos

- Las GPUs pueden combinarse con otras arquitecturas. Las posibles combinaciones de CUDA con las distintas herramientas son las siguientes:
  - MultiGPU + Multicores:
    - CUDA + OpenMP
    - CUDA + Pthreads
  - MultiGPU + Cluster:
    - CUDA + MPI
  - MultiGPU + Multicores + Cluster:
    - CUDA + OpenMP + MPI
    - CUDA + Pthreads + MPI



3

Modelos híbridos

- *CUDA + OpenMP*
- *CUDA + Pthreads*
- *CUDA + MPI*
- *CUDA + OpenMP + MPI / CUDA + Pthreads + MPI*

CUDA + OpenMP

- Se suele asignar un hilo a cada GPU para su gestión y el resto de los hilos a cómputo:



- El compilador CUDA está basado en GCC. Por lo tanto, pueden utilizarse las opciones de GCC. Un programa CUDA+OpenMP se compila:

```
nvcc -Xcompiler -fopenmp programa.cu
```



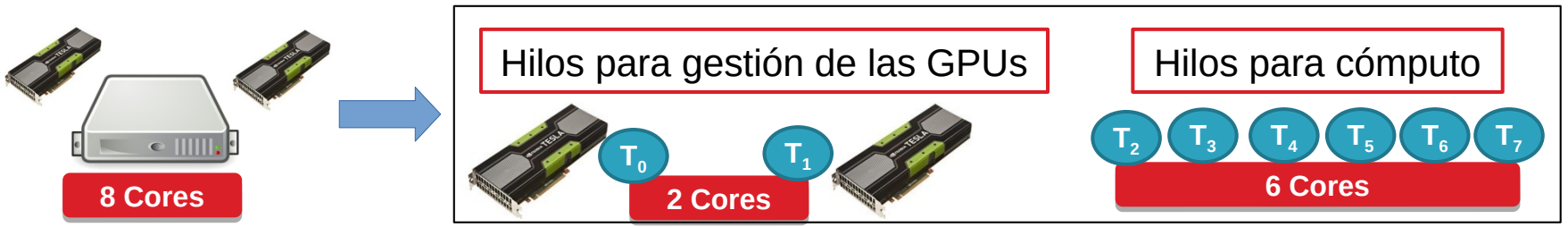
3

Modelos híbridos

- *CUDA + OpenMP*
- *CUDA + Pthreads*
- *CUDA + MPI*
- *CUDA + OpenMP + MPI / CUDA + Pthreads + MPI*

CUDA + Pthreads

- Con Pthreads se sigue la misma estrategia que OpenMP.



- Un programa CUDA+Pthreads se compila:

```
nvcc -pthread programa.cu
```



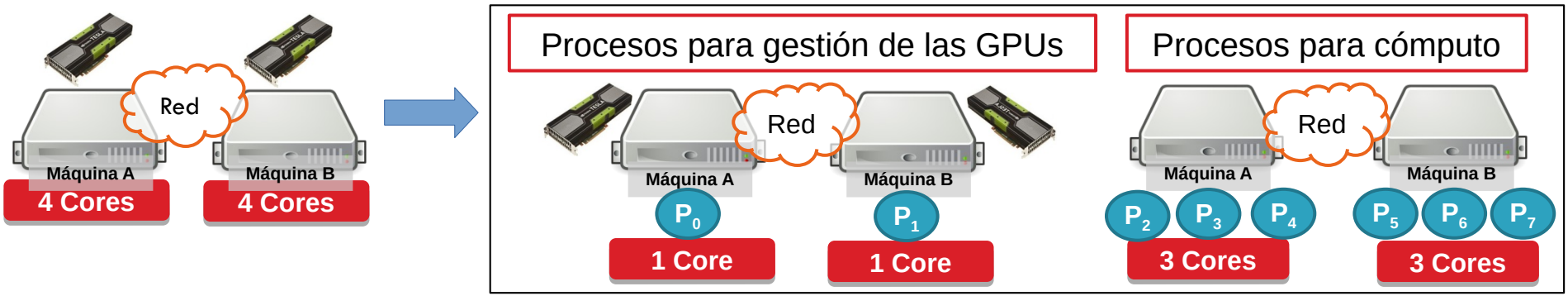
3

Modelos híbridos

- *CUDA + OpenMP*
- *CUDA + Pthreads*
- *CUDA + MPI*
- *CUDA + OpenMP + MPI / CUDA + Pthreads + MPI*

CUDA + MPI

- Siguiendo la idea de memoria compartida, se asigna un proceso de gestión por GPU y el resto a cómputo:



- Puede compilarse con `nvcc` o `mpicc`. La forma más simple es hacerlo con nvcc. Un programa CUDA+MPI se compila:

```
nvcc -I$MPI_PATH/include -L$MPI_PATH/lib -lmpi programa.cu
```

- Luego, se ejecuta con `mpirun`



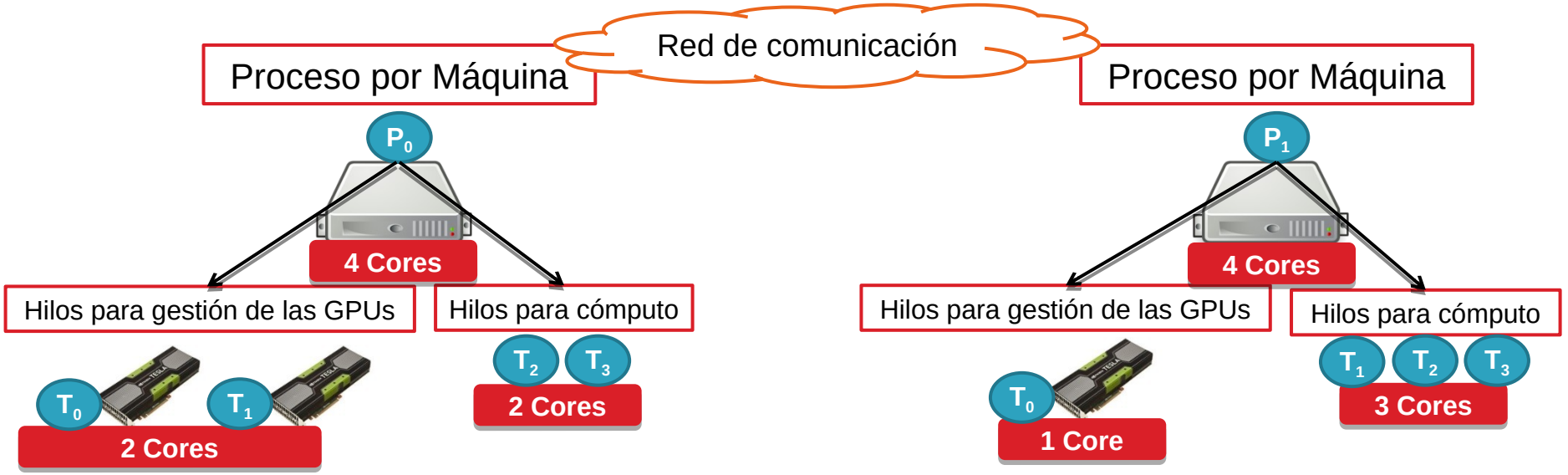
3

Modelos híbridos

- *CUDA + OpenMP*
- *CUDA + Pthreads*
- *CUDA + MPI*
- *CUDA + OpenMP + MPI / CUDA + Pthreads + MPI*

CUDA + OpenMP + MPI
CUDA + Pthreads + MPI

- Se asigna un proceso por máquina del cluster y este proceso crea los hilos.
- Luego se sigue la estrategia de memoria compartida (se crea y asigna un hilo de gestión por cada GPU y el resto a cómputo):



CUDA + OpenMP + MPI
CUDA + Pthreads + MPI

- La forma más sencilla de compilar es utilizando `nvcc`:
  - Para compilar Cuda+OpenMP+MPI:

```
nvcc -Xcompiler -fopenmp -I$MPI_PATH/include -L$MPI_PATH/lib -lmpi  programa.cu
```

  - Para compilar Cuda+Pthreads+MPI:

```
nvcc -I$MPI_PATH/include -L$MPI_PATH/lib -lmpi programa.cu
```

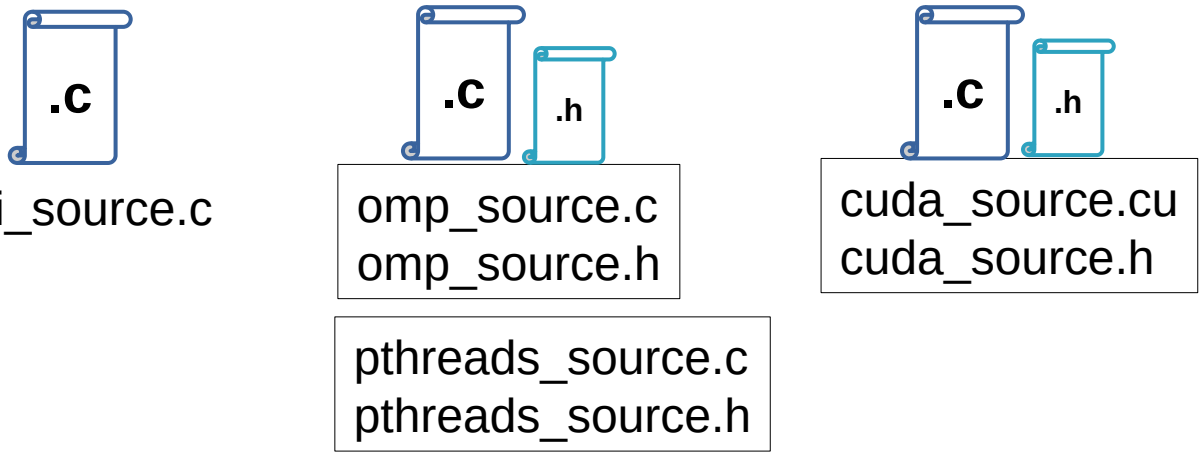
- Luego, se ejecuta con `mpirun`



4 Modularidad

Modularidad

- Lo anterior asume un único archivo `.cu` que integra todas las herramientas.
- Para lograr mayor **modularidad**, es conveniente ubicar el código de cada herramienta en archivos separados.



Modularidad
CUDA + OpenMP

- Compilación con un **único compilador**:

```
nvcc -Xcompiler -fopenmp omp_source.c cuda_source.cu
```

- **Compilación separada**:
  - **Compilar** cada herramienta con su compilador:
    - **OpenMP:** `gcc -fopenmp -c omp_source.c -o omp_source.o`
    - **CUDA:** `nvcc -c cuda_source.cu -o cuda_source.o`
  - **Linkear** los archivos de salida (`.o`) con un compilador:

```
gcc cuda_source.o omp_source.o -L$CUDA_PATH/lib64 -lgomp -lcudart
```

Modularidad
CUDA + Pthreads

- Compilación con un **único compilador**:

```
nvcc pthreads_source.c cuda_source.cu
```

- **Compilación separada**:
  - **Compilar** cada herramienta con su compilador:
    - **Pthreads:** `gcc -pthread -c pthreads_source.c -o pthreads_source.o`
    - **CUDA:** `nvcc -c cuda_source.cu -o cuda_source.o`
  - **Linkear** los archivos de salida (`.o`) con un compilador:

```
gcc cuda_source.o pthreads_source.o -L$CUDA_PATH/lib64 -pthread -lcudart
```

Modularidad
CUDA + OpenMP + MPI

- Compilación con un **único compilador**:

```
nvcc -Xcompiler -fopenmp -I$MPI_PATH/include -L$MPI_PATH/lib -lmpi mpi_source.c omp_source.c
cuda_source.cu
```

- **Compilación separada**:
  - **Compilar** cada herramienta con su compilador:
    - **MPI:** `mpicc -c mpi_source.c -o mpi_source.o`
    - **OpenMP:** `gcc -fopenmp -c omp_source.c -o omp_source.o`
    - **CUDA:** `nvcc -c cuda_source.cu -o cuda_source.o`
  - **Linkear** los archivos de salida (`.o`) con un compilador:

```
mpicc mpi_source.o omp_source.o cuda_source.o -L$CUDA_PATH/lib64 -lcudart -lgomp
```

Modularidad
CUDA + Pthreads + MPI

- Compilación con un **único compilador**:

```
nvcc -I$MPI_PATH/include -L$MPI_PATH/lib -lmpi mpi_source.c pthreads_source.c
cuda_source.cu
```

- **Compilación separada**:
  - **Compilar** cada herramienta con su compilador:
    - **MPI:** `mpicc -c mpi_source.c -o mpi_source.o`
    - **Pthreads:** `gcc -pthread -c pthreads_source.c -o pthreads_source.o`
    - **CUDA:** `nvcc -c cuda_source.cu -o cuda_source.o`
  - **Linkear** los archivos de salida (.o) con un compilador:

```
mpicc mpi_source.o pthreads_source.o cuda_source.o -L$CUDA_PATH/lib64 -lcudart
```



5 Distribución de carga

Distribución de carga

- Una arquitectura híbrida es una arquitectura heterogénea: Las unidades de procesamiento son diferentes, con distintas potencias de cómputo.
- Cuando tenemos un problema con determinada carga de trabajo es necesario que cada unidad de procesamiento reciba parte de la carga de trabajo proporcional a su potencia de cómputo.
- Se debería balancear la carga de trabajo de manera que todas las unidades de procesamiento terminen el trabajo “casi al mismo tiempo” y ese tiempo debería ser el mínimo (el que maximice el rendimiento).
- Se debe tener en cuenta que el problema de distribución de carga balanceada es **NP-Completo**.
- Una solución a este problema no puede automatizarse. Sin embargo, existen varias estrategias para permitir un balance de carga aceptable.

